Loops, muxes and storage

Use these notes to connect the Verilog you wrote to the circuit it describes. The examples cover vector loops, carry wiring, population count, mux selection, operators, falling-time counters and latches.

Where this fits in your sheet

Entries 7, 13, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 36, 38. Day 1, Day 2, Day 3.

Entry numbers follow the tracker. Day labels in older filenames refer to when the notes were written; use the entry numbers to find the matching problem.

Pages	What to read
3-6	Generate and procedural loops; vector reversal, carry wiring and population count
7	Mux operators and the two levels of a 4-to-1 mux
8	Resetting the Lemmings fall counter and handling long falls
9-12	Reset types, operator choices and intentional latch storage

Reviewed against the saved tracker and available code on 7 September 2026. Earlier submission dates inside these notes are historical records.

1. Verified problem set and question map

This map covers the previous twelve verified Day 2 problems and the nineteen additional successes from this update. Peach rows contain a question answered in this PDF; gray rows had no code question.

No.	HDLBits ID	Tracker problem	Question	Core point
19	Dff8r	DFF with reset	No question	Synchronous reset
20	Wire	Simple wire	No question	Direct connection
21	Bugs_nand3	NAND	No question	Port order + inversion
22	Fsm2	FSM 2 (async reset)	No question	OFF/ON Moore FSM
23	Vector100r	Vector reversal 2	Answered	genvar vs integer
24	Dff8p	DFF with reset value	No question	Negedge; reset 8'h34
25	Wire4	Four wires	No question	Named wire mapping
26	Popcount255	255-bit population count	Answered	initial, = vs ==
27	Fsm2s	FSM 2 (sync reset)	No question	Synchronous reset
28	Dff8ar	DFF async reset	No question	Reset in sensitivity list
29	Bugs_mux4	Mux	Answered	Selector hierarchy
30	Adder100i	100-bit adder 2	Explained	Ripple-carry generate
31	Notgate	Inverter	Answered	~ versus !
33	Dff16e	DFF with byte enable	No question	Per-byte enables
34	Bcdadd100	100-digit BCD adder	No question	100 generated stages
35	Andgate	AND gate	No question	Basic bitwise AND
36	Bugs_addsubz	Add/sub	Answered	& versus && in if
37	m2014_q4h	Wire	No question	Direct connection
38	m2014_q4a	D Latch	Answered	Latch inference
39	Fsm3onehot	One-hot transitions 3	No question	One-hot next state
40	Norgate	NOR gate	No question	Two-input NOR
41	m2014_q4i	GND	No question	Constant zero
42	m2014_q4b	DFF	No question	Async reset DFF
43	Fsm3	FSM 3 (async reset)	No question	Async reset FSM
44	Xnorgate	XNOR gate	No question	Bitwise XNOR
45	m2014_q4e	NOR	No question	Two-input NOR
46	m2014_q4c	DFF	No question	Sync reset DFF
47	Always_case	Case statement	No question	Combinational case
48	Fsm3s	FSM 3 (sync reset)	No question	Sync reset FSM
49	Wire_decl	Declaring wires	No question	Intermediate nets
50	m2014_q4f	Another gate	No question	Gate expression

Previously unanswered Day 1 questions already closed

Lemmings4	Why does the fall counter not need another reset?
Bugs_mux2	Why are bitwise | and & correct, while logical || and && are not?

2. genvar: what it is, and what it is not

Question

Why does `genvar` make sense in a generate-for loop, and why can a procedural loop use `integer i` instead?

Answer

A `genvar` is an elaboration-only index. The elaborator substitutes each constant index and creates a separate structural item or module instance. The genvar does not become a changing signal in the circuit.

```
genvar i;
generate
    for (i = 0; i < 4; i = i + 1) begin : reverse_bits
        assign out[i] = in[3-i];
    end
endgenerate
```

The elaborated structure

```
assign out[0] = in[3];
assign out[1] = in[2];
assign out[2] = in[1];
assign out[3] = in[0];
```

How the loop runs

Do not equate procedural for with a clock-time loop. A procedural loop inside always @(*) executes in the same simulation time step whenever the process runs. With constant bounds, synthesis normally unrolls it into hardware too. The distinction is structural elaboration semantics versus procedural block semantics.

Property	Generate-for	Procedural for
Where	Module/generate scope	Inside always, initial, function, or task
Index	genvar	integer or another integral variable
When semantics apply	Elaboration	When the process executes
Natural use	Instances, named scopes, separate assigns	Algorithmic assignment to variables
Constant-bound synthesis	Creates repeated structure	Usually unrolled into repeated logic

3. Vector100r: both loop styles are valid

The accepted solution uses a procedural loop. This is valid because out is assigned inside a combinational always block. The loop visits all 100 positions in one process evaluation; it does not wait 100 clocks.

```
integer i;
always @(*) begin
    for (i = 0; i < 100; i = i + 1)
        out[99-i] = in[i];
end
```

Structural alternative

```
genvar i;
generate
    for (i = 0; i < 100; i = i + 1) begin : reverse_bit
        assign out[i] = in[99-i];
    end
endgenerate
```

Both descriptions reduce to fixed wiring. No storage or run-time selector is required. The two index forms describe the same reversal when i covers 0 through 99.

i	Procedural form	Generate form	Connection
0	out[99] = in[0]	out[0] = in[99]	Two ends are paired
1	out[98] = in[1]	out[1] = in[98]	Next inward pair
98	out[1] = in[98]	out[98] = in[1]	Next inward pair
99	out[0] = in[99]	out[99] = in[0]	Two ends are paired

Choice rule

Use a procedural loop when one combinational or sequential process naturally owns the assignments. Use generate-for when stamping out instances, named scopes, or separate structural connections.

4. Adder100i: genvar stamps out a carry chain

Each loop iteration creates one full adder. carry is a wire because it connects continuously driven module outputs to inputs; it stores no clocked value. There are 101 carry nodes: the carry-in plus one output from each of the 100 stages.

```
wire [100:0] carry;
assign carry[0] = cin;

genvar i;
generate
    for (i = 0; i < 100; i = i + 1) begin : fa_stage
        fa dut (
            .a   (a[i]),
            .b   (b[i]),
            .cin (carry[i]),
            .sum (sum[i]),
            .cout(carry[i+1])
        );
    end
endgenerate

assign cout = carry[100:1];
```

Stage	Inputs	Outputs	Meaning
0	a[0], b[0], carry[0]=cin	sum[0], carry[1]	Least-significant bit
i	a[i], b[i], carry[i]	sum[i], carry[i+1]	Generic generated stage
99	a[99], b[99], carry[99]	sum[99], carry[100]	Most-significant bit

Why cout = carry[100:1] works

HDLBits defines cout[i] as the carry output of bit i. Stage i writes carry[i+1], so carry[100:1] aligns exactly with cout[99:0].

- Keep carry[0] tied to cin; otherwise stage 0 has no defined carry-in.

- Use carry[i+1] for each stage output; carry[i] is that stage's input.

- Name the generate block so tools show readable hierarchy such as fa_stage[37].dut.

- The loop creates 100 full adders at elaboration; it does not reuse one full adder for 100 cycles.

5. Popcount255: initialization belongs inside combinational logic

Question

Why does an `initial begin` block not work for setting out to zero, and when should a procedural loop be used instead of generate?

Answer

An initial block runs once at simulation time zero. A population count must be recomputed every time the 255-bit input changes. Therefore out receives its default zero at the start of the combinational always block, before the loop adds the set bits.

```
always @(*) begin
    out = 8'd0;
    for (i = 0; i < 255; i = i + 1) begin
        if (in[i])
            out = out + 8'd1;
    end
end
```

Code idea	Meaning	Why it matters
out = 8'd0	Assignment	Reinitializes the accumulator on every evaluation
out == 8'd0	Comparison	Computes true/false and does not assign out
Blocking =	Immediate procedural update	Each iteration sees the previous iteration's sum
Nonblocking <=	Deferred update	Iterations tend to read the same old out
8-bit out	Range 0 through 255	Exactly enough for the maximum count

Procedural loop versus generate here

The procedural loop is natural because one combinational process owns one accumulator. Synthesis can build an adder network from the loop. A generate solution would need explicit partial sums or an adder tree; it must not create 255 drivers on one out vector.

Small cleanup

The test if (in[i] && 1'b1) is equivalent to if (in[i]). The shorter condition states the intent directly.

6. Mux operators and selectors

Entry 13: keep the whole eight-bit value

The HDLBits mux selects a when sel=0 and b when sel=1. Repeat sel eight times to make an eight-bit mask. Bitwise AND and OR then select every bit of the chosen bus. Logical operators would reduce each bus to a one-bit truth value.

```
assign out = ({8{~sel}} & a) | ({8{sel}} & b);
// The conditional form is shorter:
assign out = sel ? b : a;
```

sel	a mask result	b mask result	out
0	a	0	a
1	0	b	b

Entry 29: build the four-way mux in two levels

Use sel[0] in both first-level muxes. It chooses within the a/b pair and within the c/d pair. Then sel[1] chooses which pair reaches out. The intermediate signals must be eight bits wide.

```
wire [7:0] lower_pair, upper_pair;
mux2 m0(sel[0], a, b, lower_pair);
mux2 m1(sel[0], c, d, upper_pair);
mux2 m2(sel[1], lower_pair, upper_pair, out);
```

sel[1:0]	00	01	10	11
out	a	b	c	d

If the pair outputs were single-bit wires, most of each bus would be lost before the final mux. Distinct instance names also avoid the name collisions in the supplied buggy code.

Sources: https://hdlbits.01xz.net/wiki/Bugs_mux2 and https://hdlbits.01xz.net/wiki/Bugs_mux4

7. Reset the counter when a fall ends

Entry 7: why is another reset unnecessary?

On the landing edge, the current state still describes the fall. The next-state logic can use the old fall count to choose walking or splatter while the clocked block schedules count back to zero. The reset takes effect after the decision, so the measured length is still available when it is needed.

Moment	Count use	Count update
Falling	Measure consecutive falling cycles	Increment, with saturation
Landing	Use old count to decide survival	Clear for a later fall
Reset	Discard the old measurement	Clear immediately
Splattered	Count no longer affects behavior	Death remains permanent

Two details the earlier excerpt left out

A five-bit counter wraps after 31. A long fall could then look short, so stop incrementing once the counter reaches 21. The task distinguishes falls of at most 20 cycles from falls longer than 20; it does not need the exact length after that threshold.

Also decide which edge counts the first falling cycle. The monitor below includes the first edge with ground=0, including the edge that leaves walking or digging. A design that increments only while already in a falling state must account for that different starting point.

```
// Fall-length monitor; combine with the existing movement FSM.
always @(posedge clk or posedge areset) begin
    if (areset)
        count <= 5'd0;
    else if (ground)
        count <= 5'd0;
    else if (count < 5'd21)
        count <= count + 5'd1;
end
// While in a falling state, on landing:
// next_state = (count > 5'd20) ? SPLATTER : WALK_SAVED_DIR;
```

SPLATTER must remain terminal until reset. Clearing the counter does not revive the lemming. With this counting convention, 20 low-ground samples followed by landing survive; 21 or more splatter.

This is a counter fragment, not a complete Lemmings4 submission. Source: https://hdlbits.01xz.net/wiki/Lemmings4

8. Other verified Day 2 takeaways

Problem	Revision takeaway
Dff8r	Synchronous reset is tested inside always @(posedge clk); q changes only on a rising edge.
Wire	A continuous assignment models a direct combinational connection with no storage.
Bugs_nand3	The provided module port order must match; NAND is the inverted AND result.
Fsm2	An asynchronous reset belongs in the sensitivity list; next-state logic remains combinational.
Dff8p	The DFF triggers on the falling edge and loads the exact reset value 8'h34.
Wire4	Each output is a named connection; duplicating b to x and y is intentional.
Fsm2s	A synchronous reset is checked after a clock edge; reset is not in the sensitivity list.
Dff8ar	An active-high asynchronous reset uses posedge areset alongside posedge clk.

One decision tree for future HDLBits loops

Question	If yes	Likely construct
Creating repeated instances or structural scopes?	Yes	generate-for + genvar
Assigning or accumulating inside one always block?	Yes	procedural for + integer/integral index
Must work happen over multiple clock edges?	Yes	counter/FSM/register state, not a loop alone

Final recall

genvar is a generate variable used while elaborating repeated structure. A procedural loop index controls statements inside a process. Clock-to-clock repetition requires explicit registered state.

Problem references

https://hdlbits.01xz.net/wiki/Vector100r

https://hdlbits.01xz.net/wiki/Popcount255

https://hdlbits.01xz.net/wiki/Bugs_mux4

https://hdlbits.01xz.net/wiki/Adder100i

https://hdlbits.01xz.net/wiki/Lemmings4

https://hdlbits.01xz.net/wiki/Bugs_mux2

9. Inverter: ~ versus ！

Question found in the successful Notgate code

Is ~ bitwise and ！logical?

Answer

Yes. ~ complements every bit and preserves the operand width. ！asks whether the entire operand is logically false and returns one bit. For a single known 0/1 input they give the same inverter result, but they are not interchangeable for a vector.

```
// Best expression for the one-bit HDLBits inverter
assign out = ~in;

// Width contrast for a vector
~8'b0000_0010   // 8'b1111_1101
!8'b0000_0010   // 1'b0  (the vector is nonzero/true)
!8'b0000_0000   // 1'b1  (the vector is zero/false)
```

Why both look identical for this one-bit problem

The input and output are each one bit. For a known input of 0, both operators produce 1; for a known input of 1, both produce 0. The important difference is the rule they apply: ~ works bit by bit, while ！first converts the complete operand into one truth value.

Four-state note

Unknown or high-impedance bits can propagate an unknown result. That is another reason to choose the operator by intent instead of relying only on a small 0/1 example.

Recall

Use ~ for gates and per-bit inversion. Use ！when you need a one-bit true/false test of an entire expression.

https://hdlbits.01xz.net/wiki/Notgate

10. Add/sub: why & and && both work here

Question found in the successful Bugs_addsubz code

Why do `out & 8'b11111111` and `out && 8'b11111111` select the same branch in this `if`?

Answer

The bitwise expression leaves `out` unchanged because the mask is all ones. The logical expression sees that the all-ones constant is true, so it reduces to the truth value of `out`. An `if` also converts its condition to a truth value. Therefore both test whether `out` is nonzero.

```
out &  8'hFF   == out
out && 8'hFF   == bool(out) && 1'b1
               == bool(out)

// Direct zero detector; reduction NOR is clear and width-safe.
result_is_zero = ~|out;
```

Step-by-step examples

out	out & 8'hFF	out && 8'hFF	if branch
8'h00	8'h00	1'b0	false
8'h02	8'h02	1'b1	true
8'hA0	8'hA0	1'b1	true

Do not generalize this equivalence

The two operators work the same only because the right-hand operand is an all-ones value and the result is immediately used as a condition. With `8'h0F`, bitwise & masks the upper four bits, while logical && discards bit positions and returns only one bit.

Best expression for the intent

If the output should be high exactly when `out` is zero, write `result_is_zero = ~|out;` or `result_is_zero = (out == 8'b0);`. The intent is visible immediately.

https://hdlbits.01xz.net/wiki/Bugs_addsubz

11. D Latch: what 'latch inference' means

Question found in the successful Exams/m2014_q4a code

What does latch inference mean for these assignment lines, and why does the synthesis tool report a latch?

Answer

When `ena` is 0, the code requires `q` to retain its previous value. Retaining a value requires storage. Because the control is a level signal rather than a clock edge, synthesis creates a level-sensitive D latch. The warning is expected here because the HDLBits problem intentionally asks for a latch.

```
// Intentional latch: no assignment while ena is low.
always @(*) begin
    if (ena)
        q <= d;
end

// This says the same thing, but q <= q is redundant:
// else q <= q;
```

How the inferred hardware behaves

ena	Latch behavior	Meaning
1	q follows d	The latch is transparent while enabled.
0	q holds its old value	The latch is closed and stores the last d value.

Why this matters in ordinary combinational logic

In a block meant to be purely combinational, every output must be assigned on every path. If one path leaves an output unchanged, a latch can be inferred accidentally. Here it is intentional; elsewhere it is often a bug. In SystemVerilog, `always_latch` can state the intent more explicitly.

Latch versus flip-flop

A latch is **level-sensitive**: while enabled, output can respond to changes in `d`. A flip-flop is **edge-triggered**: output changes only at its specified clock edge.

Recall

No assignment (or `q <= q`) means 'remember the old value.' Remembering under a level enable is exactly the job of a latch.

https://hdlbits.01xz.net/wiki/Exams/m2014_q4a